Audit initial de DropZone

Projet audité : DropZone (plateforme de drops de sneakers) **Stack :** API FastAPI (Python) + front React (TypeScript, Vite) **État audité :** dépôt de départ, avant toute modification (partie 1 du TP)

Légende : ✅ OK · ⚠️ partiel · ❌ absent · 🔎 à vérifier

Git

Point vérifié	Note	Constat
Dépôt GitHub existant et accessible	✅	Dépôt créé à partir du template `dropzone-starter`, partagé avec `hellodamien`
`.gitignore` adapté à la stack	✅	Couvre `.env`, `.venv/`, `__pycache__/`, `node_modules/`, `dist/`, caches (`.pytest_cache/`, `.ruff_cache/`) et rapports (`coverage/`, `playwright-report/`). `.env.example` est explicitement autorisé. `git ls-files | grep -i env` ne renvoie que `frontend/src/vite-env.d.ts` (fichier de types, légitime) : aucun `.env` commité
Historique de commits lisible	🔎	À vérifier avec `git log --oneline` : les messages sont-ils explicites ou du type « fix », « wip » ?
Branche `main` protégée	❌	Aucune règle de protection : un push direct est possible. À traiter dans cette partie 1

Build et lancement

Point vérifié	Note	Constat
L'application démarre en une seule commande documentée	⚠️	Le README documente le lancement, mais il faut deux processus séparés : l'API (venv Python + `uvicorn`) et le front (`npm run dev`). Aucun Docker ni Compose. Il faut aussi installer Python et Node sur le poste
Les dépendances sont listées	✅	Fichiers de dépendances présents côté API (Python) et côté front (`package.json`) : l'installation fonctionne à partir d'eux
Aucun chemin absolu codé en dur	🔎	À vérifier avec une recherche dans le code (voir la commande en fin de document)

Tests

Point vérifié	Note	Constat
Des tests existent	✅	Backend : `backend/tests/conftest.py` et ses tests. Front : `Countdown.test.tsx`, `DropDetailPage.test.tsx`, `frontend/src/test/fixtures.ts`
Les tests s'exécutent en une seule commande	⚠️	Deux commandes séparées (pytest pour l'API, Vitest pour le front), aucun point d'entrée unique ni exécution automatique. La couverture est incomplète : pas de tests sur les commandes (`test_orders.py`), le composant `DropCard`, ni de tests de bout en bout

Déploiement

Point vérifié	Note	Constat
Méthode de déploiement connue et documentée	❌	Aucune : pas d'image Docker, pas de pipeline, pas de procédure dans le README
Un environnement de staging existe	❌	Aucun environnement : l'application ne tourne que sur un poste de développeur

Monitoring

Point vérifié	Note	Constat
L'URL de production est surveillée	❌	Pas de production, donc aucune sonde. L'API expose `/health` et `/health/ready`, mais rien ne les interroge
Les logs sont accessibles	⚠️	Logs en texte brut produits par `backend/app/logging_config.py`, visibles seulement dans le terminal : non structurés, non centralisés, non conservés

Bilan

Total : 4 ✅ · 3 ⚠️ · 4 ❌ · 2 🔎 (sur 13 points)

Ce qui est présent

- Une application fonctionnelle en local (API et front)
- Un `.gitignore` sérieux et des dépendances déclarées
- Des tests unitaires de départ (API et front)
- Des endpoints de santé (`/health`, `/health/ready`)

Ce qui manque

- Conteneurisation (Dockerfiles, Compose, `.env.example`)
- Intégration continue (aucun dossier `.github/`, donc aucun workflow)
- Déploiement continu, publication de versions, procédure de retour arrière
- Supervision : métriques, logs structurés, alertes
- Documentation de contribution (`CONTRIBUTING.md`, `CHANGELOG.md`, modèle de PR) et versions taguées

Ce qui est risqué

- `main` **non protégée** : un code cassé peut être poussé directement, sans relecture ni test
- **Aucune vérification automatique** : une régression (par exemple vendre deux fois la dernière paire) peut passer inaperçue
- **Livraison non reproductible** : le lancement dépend de l'environnement du poste (versions de Python et de Node)
- **Aucune alerte** : une panne ne serait détectée que par les utilisateurs

Mes 3 priorités d'amélioration

1. **Protéger** `main` et imposer pull requests et commits conventionnels (partie 1), puis rendre la CI obligatoire (partie 3).
2. **Conteneuriser l'application** pour qu'une seule commande la lance, sans rien installer d'autre que Docker (partie 2).
3. **Automatiser la livraison et la supervision** : CI, CD avec versions et retour arrière, métriques, logs structurés et sonde avec alerte (parties 3 à 5).

Réponses aux questions de la partie 1

Voir QUESTIONS-PARTIE-1.md (ou à copier ici).

Commandes utilisées pour cet audit

```bash
ls -a                                   # contenu de la racine
cat .gitignore                          # couverture du .gitignore
git ls-files | grep -i env              # fichiers d'environnement suivis par Gi
ls -a .github                           # présence d'une CI : dossier absent
git log --oneline                       # lisibilité de l'historique (à complète
grep -rnE "C:\\\\|/Users/|/home/" backend frontend --include="*.py" --include="
```